

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical - IX

Roll No.: T010	Name: Sagar Knadari
Class: TYIT	Batch: 1
Date of Assignment: 18-08-26	Date/Time of Submission: 18-08-26

PART A – Theory

Practical File – Theory

A. CRUD Operations using ASP.NET Core Web API and Entity Framework Core

CRUD stands for **Create, Read, Update, and Delete**. These are the basic operations used to manage data in a database.

- **Create** – Add new records to the database.
- **Read** – Retrieve existing records.
- **Update** – Modify an existing record.
- **Delete** – Remove a record from the database.

Entity Framework Core (EF Core) is an Object-Relational Mapper (ORM) that allows ASP.NET Core applications to work with databases using C# classes and objects instead of writing SQL queries for every operation.

Common HTTP methods used for CRUD:

Operation HTTP Method

Create	POST
Read	GET
Update	PUT
Delete	DELETE

B. Factorial API and Postman

A **Factorial** of a number n is the product of all positive integers from 1 to n .

Formula:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 1$$

Example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

An **API endpoint** can be created to accept a number, calculate its factorial, and return the result.

Postman is a tool used to test Web APIs by sending HTTP requests and viewing their responses without creating a separate user interface.

Example:

Request:
GET /api/factorial/5

Response:
{
 "number": 5,
 "factorial": 120
}

Key Concepts to Remember

CRUD → POST, GET, PUT, DELETE

Factorial API → Accept number → Calculate → Return JSON

Postman → Send request → Test API → View response

code:-

student.cs

namespace Student_API_Core_Web_API.Models

{

```
public class Student
{
    public int Id { get; set; }

    public string Name { get; set; }

    public int Age { get; set; }

    public string Course { get; set; }
}
}
```

appdb.contextcs

```
using Microsoft.EntityFrameworkCore;
using Student_API_Core_Web_API.Models;

namespace Student_API_Core_Web_API.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) {}

        public DbSet<Student> Students { get; set; }
    }
}
```

program.cs

```
using Microsoft.EntityFrameworkCore;

using Student_API_Core_Web_API.Data;


var builder = WebApplication.CreateBuilder(args);


// Add services to the container

builder.Services.AddControllers();


// Register DbContext with SQL Server

builder.Services.AddDbContext<AppDbContext>(options =>

    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));


// Add Swagger

builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();


var app = builder.Build();


app.UseSwagger();

app.UseSwaggerUI();


app.UseHttpsRedirection();

app.UseAuthorization();
```

```
app.MapControllers();
```

```
app.Run();
```

```
studentcontroller.cs
```

```
using Microsoft.AspNetCore.Mvc;
```

```
using Student_API_Core_Web_API.Data;
```

```
using Student_API_Core_Web_API.Models;
```

```
namespace Student_API_Core_Web_API.Controllers
```

```
{
```

```
    [ApiController]
```

```
    [Route("api/[controller]")]
```

```
    public class StudentsController : ControllerBase
```

```
    {
```

```
        private readonly ApplicationDbContext _context;
```

```
        public StudentsController(ApplicationDbContext context)
```

```
        {
```

```
            _context = context;
```

```
        }
```

```
        [HttpPost]
```

```
        public IActionResult AddStudent(Student student)
```

```
        {
```

```
            _context.Students.Add(student);
```

```
        _context.SaveChanges();

        return Ok(student);
    }

// UPDATE

[HttpPut("{id}")]
public IActionResult UpdateStudent(int id, Student student)
{
    var data = _context.Students.Find(id);

    if (data == null) return NotFound();

    data.Name = student.Name;

    data.Age = student.Age;

    data.Course = student.Course;

    _context.SaveChanges();

    return Ok(data);
}

// DELETE

[HttpDelete("{id}")]
public IActionResult DeleteStudent(int id)
{
    var student = _context.Students.Find(id);

    if (student == null) return NotFound();
}
```

```
        _context.Students.Remove(student);  
  
        _context.SaveChanges();  
  
        return Ok();  
    }  
}  
}
```

output:-

The screenshot shows a REST client interface with a 'Request body' tab selected. The media type is set to 'application/json'. The request body contains a JSON object: `{ "id": 0, "name": "sagar", "age": 20, "course": "bsc-it" }`. Below the request body is a blue 'Execute' button. The 'Responses' tab is also visible, showing a table with one response: a 200 status code with the description 'OK' and no links.

Code	Description	Links
200	OK	No links

Curl

```
curl -X 'POST' \
  'https://localhost:44300/api/Students' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 0,
    "name": "sagar",
    "age": 20,
    "course": "BCa"
  }'
```

Request URL

https://localhost:44300/api/Students

Server response

Code	Details
200	Response body <pre>{ "id": 0, "name": "sagar", "age": 20, "course": "BCa" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 18 Aug 2020 03:57:30 GMT server: Microsoft-IIS/10.0 x-powered-by: ASP.NET</pre>

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:44300/api/Students' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 0,
    "name": "sagar",
    "age": 20,
    "course": "BCa"
  }'
```

Request URL

https://localhost:44300/api/Students

Server response

Code	Details
200	Response body <pre>{ "id": 0, "name": "sagar", "age": 20, "course": "BCa" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 18 Aug 2020 03:57:30 GMT server: Microsoft-IIS/10.0 x-powered-by: ASP.NET</pre>

Responses


```
curl -X 'DELETE' \
  'https://localhost:44300/api/Students/1' \
  -H 'accept: */*'

Request URL
https://localhost:44300/api/Students/1

Server response

Code    Details
404     Error: response status is 404
Undocumented

Response body
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
  "title": "Not Found",
  "status": 404,
  "traceId": "00-e4d8a6cd3d1ad50d69006d297efeb710-598a4d51e953e686-00"
}

Response headers
content-type: application/problem+json; charset=utf-8
date: Tue, 18 Aug 2026 04:08:39 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET

Responses

Code    Description    Links
200     OK             No links
```

PART B – Questions

- Develop CRUD operations using ASP.NET Core Web API and Entity Framework Core.
- Create an API endpoint to calculate Factorial of a Number and test it using Postman.

PART C – Curiosity Question

- If a student record is stored in a database, how can our Web API add, display, edit, and delete that record?

- 2. Can we perform database operations without writing SQL queries every time?**
- 3. What happens when we send a GET, POST, PUT, or DELETE request to an API?**
- 4. Can an API calculate something like $5! = 120$ without having a webpage?**
- 5. How can we test an API if we don't have a frontend application?**
- 6. If I give an API the number 10, can it return the factorial directly in JSON?**